Device-to-app networking

MQTT, CoAP, and TLS on small devices

Dinu-Ștefan Rusu
FILS, Universitatea Politehnica București · Spring 2027

TODAY

What you should leave with

Choose a transport

Why polling a sensor is wrong, what the four push options are, and where MQTT fits.

Use MQTT properly

Topics and wildcards, QoS levels, retained messages, last will, keep-alive and sessions.

Secure a small device

TLS with certificates or pre-shared keys, one identity per device, and credential rotation.

Ship an update

Two partitions, a signature check and a rollback, so a bad image does not brick the board.

PART 1 · PUSHING

Getting a reading off the device

Why polling is the wrong shape, and the four alternatives

Why polling a sensor is the wrong shape

A sensor produces a value when something happens. Polling asks on a fixed schedule, so the answer is either late or wasted.

Every poll pays for a full request: a socket, headers, a handler. Several hundred bytes to be told nothing changed.

The direction is wrong too. The server cannot reach a device that only ever asks questions.

The cost is the radio

On a battery the bytes are not the problem. Waking a Wi-Fi radio to hear that nothing changed is the worst trade a device can make, and it makes it every interval, forever.

Polling scales with the clock. Events scale with reality. A device that publishes when it has something to say sends fewer packets, reports faster, and sleeps in between.

PUSHING

Four ways to push, and when each one fits

TRANSPORT	DIRECTION	RUNS OVER	REACH FOR IT WHEN
WebSocket	full duplex	one TCP socket	both sides talk: chat, presence, editing
SSE	server to client	one long HTTP response	a one-way feed, with reconnect for free
Long polling	server to client	ordinary requests	a restrictive network blocks the rest
MQTT	publish, subscribe	TCP, or WebSocket	devices: tiny frames, QoS, a last will

MQTT is the one you meet on hardware. A broker fans one reading out to every subscriber, and a two-byte fixed header suits a part with 520 KB of SRAM.

Where MQTT fits: a broker in the middle

The device publishes

It connects outward to one address, sends a reading on a topic, and knows nothing else about the system.

The broker fans out

The only component everyone connects to. It matches topics to subscribers, keeps retained values and queues for absent clients.

Subscribers receive

Your Flutter app, a dashboard and a database can all take the same reading without the device changing a line.

The device never learns who is listening. That is the whole point of a broker: you add a consumer by subscribing, not by reflashing a board that is glued to a wall.

PART 2 · MQTT

The protocol in detail

Topics, quality of service, retained state, and what a session holds

The broker, the client, and the connection

MQTT stands for Message Queuing Telemetry Transport. It is an OASIS standard, in two versions you will meet: 3.1.1 and 5.0.

Plain TCP on port 1883, TLS on port 8883, and a WebSocket tunnel where only HTTP gets out.

A client opens with a CONNECT packet: a client identifier, optional credentials, a keep-alive and the will. The broker answers CONNACK.

On the wire

The fixed header is two bytes: a control byte and a length. Everything else is a packet type: PUBLISH, SUBSCRIBE, PINGREQ, DISCONNECT.

The client identifier is a name, not an identity. Anyone can claim it, and a second client using the same one disconnects the first. Authentication is the broker's job: Part 5.

Topics and wildcards

```
mec/lab/room1/temp     a topic. Publishers always use a full topic.
mec/lab/+/temp         '+' matches exactly one level: every room
mec/lab/#              '#' matches the rest. Last position only
mec/+/room1/#          both, in one filter
$SYS/broker/uptime     broker statistics, not matched by a root '#'
```

Topics are UTF-8 strings split by `/`. Levels are case sensitive, there is no schema, and the broker creates a topic the moment somebody publishes to it. Wildcards are valid in a subscription only.

Identity goes in the topic, data goes in the payload. A subscriber should be able to select what it wants with a wildcard, instead of receiving everything and parsing its way to the right rows.

QoS 0, 1 and 2

LEVEL	GUARANTEE	PACKETS	USE IT FOR
QoS 0	At most once	PUBLISH	a reading you resend in two seconds
QoS 1	At least once	PUBLISH, PUBACK	telemetry, if duplicates are harmless
QoS 2	Exactly once	a four-packet handshake	an irreversible command, little else

QoS 1 with an idempotent payload is the right default. The level delivered is the lower of the two, publisher and subscriber. Put a sequence number in the payload and let the receiver drop repeats.

What QoS costs, and what it does not buy

QoS is per hop. It covers device to broker, and broker to phone, as two separate promises. The publisher never learns that the phone received anything.

Above QoS 0 the client stores the message until it is acknowledged, and has to stay awake for that acknowledgement. Both cost a device that wants to sleep.

End to end

If you need to know the app got the value, the app has to say so on another topic. A sequence number gives gap detection, which no QoS level provides.

Delivery guarantees stop at the broker. Make a missing reading visible and recoverable in the application, instead of assuming a protocol flag removed the problem.

Retained messages and the last will

RETAINED

The broker keeps the last message published with the retain flag, one per topic, and hands it to every new subscriber immediately. Publish an empty retained payload to clear it. Use it for state, such as the last reading or a configuration value, never for events.

LAST WILL

Set in CONNECT: a topic, a payload, a QoS and a retain flag. The broker publishes it when the connection drops without a DISCONNECT packet, which covers a crash, a flat battery and a keep-alive timeout.

Together they give you a status topic the app can trust. Publish a retained `online` on connect, register a retained `offline` as the will, and the app has a live presence indicator without writing any presence code.

Keep-alive, sessions and reconnects

Keep-alive is a number of seconds in CONNECT. The client sends something inside that window, a PINGREQ if it has nothing else, and the broker closes the connection after one and a half intervals of silence.

A long keep-alive delays detection of a dead device. A short one keeps the radio busy. Pick it from how fast the app must notice.

Reconnect with the same identifier and a clean start of false, and the broker restores the subscriptions and any queued messages.

Battery first

A session held open exists to receive commands quickly. If the device only publishes, connect, send, disconnect and sleep. The handshake is paid once; the keep-alive is paid forever.

MQTT 5, in one slide

Reason codes

Every acknowledgement carries a code, so a refused connection or a denied publish says why.

Expiry intervals

A session and a message can both expire, so a queue for an absent device empties itself.

Topic aliases

A short integer replaces a long topic after the first publish. Fewer bytes per message.

Request and response

A response topic and correlation data are protocol fields, not a convention you invent.

Shared subscriptions

A group of consumers splits one topic between them, which is how the server side scales.

User properties

Arbitrary key and value pairs on a packet: the closest thing MQTT has to a header.

PART 3 · BOTH ENDS

From the board to the phone

A broker on your laptop, TinyGo publishing, Flutter subscribing

A broker on your own machine

```
sudo apt install mosquitto mosquitto-clients

# /etc/mosquitto/conf.d/lab.conf
listener 1883
allow_anonymous false
password_file /etc/mosquitto/passwd

sudo mosquitto_passwd -c /etc/mosquitto/passwd dev
mosquitto_sub -h localhost -u dev -P pass -t 'mec/lab/#' -v
mosquitto_pub -h localhost -u dev -P pass -t mec/lab/r1/temp -m '{"c":21.4}'
```

Subscribe in one terminal, publish in another, and watch the line appear. Getting that far before any board or app code exists removes half the debugging from Lab 2.

TinyGo: join the network and publish

```
radio := link.Esplink{}   // espradio
netdev.UseNetdev(&radio)
radio.NetConnect(&nl.ConnectParams{
  Ssid: ssid, Passphrase: pass})
conn, _ := net.Dial("tcp", broker)
c := mqtt.NewClient(mqtt.ClientConfig{})
var v mqtt.VariablesConnect
v.SetDefaultMQTT([]byte("mec-sensor-01"))
c.Connect(ctx, conn, &v)
fl, _ := mqtt.NewPublishFlags(
  mqtt.QoS0, false, false)   // natiu-mqtt
pv := mqtt.VariablesPublish{TopicName: t}
c.PublishPayload(fl, pv, payload)
```

Wi-Fi is a driver, not a given.
On the ESP32-C3 and the ESP32-S3 the radio comes from the espradio package. Check your target first.

Order of failure

Network first, broker second, sensor third. Print at every step: a silent board says nothing.

Flutter: connect, subscribe, decode

```
final c = MqttServerClient('10.0.2.2', 'phone-1')  // mqtt_client
  ..port = 1883
  ..keepAlivePeriod = 30
  ..onDisconnected = _scheduleReconnect;
await c.connect('dev', 'pass');
c.subscribe('mec/lab/+/temp', MqttQos.atLeastOnce);  // + = one level
c.updates!.listen((events) {
  final m = events.first.payload as MqttPublishMessage;
  final text = MqttPublishPayload.bytesToStringAsString(m.payload.message);
  _bloc.add(ReadingReceived(jsonDecode(text)));
});
```

`10.0.2.2` is the host machine seen from the Android emulator. On a real phone use the laptop's address on the lab network.

Reconnect, and the app going to background

```
Future<void> _scheduleReconnect() async {
  var delay = const Duration(seconds: 1);
  while (!_disposed && !_connected) {
    await Future.delayed(delay + _jitter());
    try { await c.connect(user, pass); } catch (_) {}
    delay = delay * 2;          // cap this at about a minute
  }
}
```

Every long-lived connection needs a reconnect policy. Backoff, jitter, a ceiling, and cancellation in dispose. The socket also dies while the app is in the background, so reconnect on resume and use a push notification for anything that must reach the user while the app is away.

BOTH ENDS

The lab broker is not the project broker

CONCERN	LOCAL MOSQUITTO, LAB 2	HOSTED BROKER, PROJECT
Reachable from	the lab network only	anywhere, over the internet
Authentication	a password file you write	an account per device, managed
Transport	plain TCP on 1883	TLS on 8883, without exception
Persistence	off unless configured	queues survive a restart
Who may subscribe	anyone holding the password	only what the ACL permits

Develop against the local one, demo against the hosted one. A plain 1883 listener is a lab tool. The moment it is reachable from outside the room it is an open microphone and an open command channel.

PART 4 · COAP

Request and response on UDP

When a datagram protocol beats a broker

CoAP: REST for constrained devices

A familiar shape

GET, POST, PUT and DELETE on URIs, with response codes that mirror HTTP. RFC 7252.

Four bytes of header

Over UDP, so there is no connection to set up. A device can wake, send one datagram, and sleep.

Reliability per message

A confirmable message is acknowledged and retransmitted. A non-confirmable one is not.

Observe

An extension that turns a resource into a subscription: the server keeps sending updates.

Block-wise transfer

A payload larger than one datagram is split into blocks.

DTLS

Security is DTLS on port 5684, with pre-shared keys or certificates.

CoAP or MQTT

QUESTION	MQTT	COAP
Shape	publish and subscribe	request and response
Transport	TCP, with TLS	UDP, with DTLS
Extra parts	a broker you must run	none, the device serves
Fan-out	the broker does it	you do it
Through NAT	an outbound connection works	hard to reach from outside
Best at	a stream of readings for many	one value on demand

This course uses MQTT, and the reason is NAT. A phone and a board behind two routers cannot address each other. Both reach a broker outbound.

PART 5 · TRUST

Identity, encryption, updates

What it takes to put a board on a real network and keep it there

TRUST

What an open broker gives away

Anyone who can reach the port can subscribe to `#` and read every reading every device publishes.

They can publish too. If the device acts on a command topic, they now own the device.

Without TLS the credentials travel in clear text inside CONNECT, so a password file on its own protects nothing.

⚠ Not for the demo

Do not forward port 1883 from your home router so the project works from the classroom. Use TLS on 8883 with an account per device, or a tunnel you control.

A broker is a command channel, not just a data feed. Treat write access with the seriousness you would give a shell on the board, because in practice that is what it is.

TLS on a device with kilobytes of RAM

TLS 1.3 finishes the handshake in one round trip. The expensive parts are the certificate chain and the buffers held while verifying it.

A record can be up to 16 KB, so a careless stack wants two buffers that size. A constrained client asks for a smaller limit.

Elliptic curve key exchange costs tens of milliseconds, and RSA costs more. The ESP32 has hardware for the symmetric work.

The clock problem

Validity is checked against the current time. A board with no real-time clock must fetch the time first, or every handshake fails with an expiry error.

Ship one root, not a trust store. The device only talks to your broker, so it needs the one authority that signed it.

TRUST

Certificates versus pre-shared keys

	X.509 CERTIFICATES	PRE-SHARED KEYS
Handshake size	a chain of one to three certs	a key identity, tens of bytes
Compute	a signature to verify each time	symmetric only, cheap
On the device	a key, a certificate, and a root	one secret per device
Retiring one	a short lifetime, or a deny list	delete the key at the broker
Forward secrecy	yes, with ephemeral exchange	only with an ephemeral PSK suite

Both work. Neither survives a shared secret in the firmware. One key compiled into an image on a thousand boards means one extracted board compromises all of them.

One identity per device

1. Generate the key on the device where the chip supports it, so the private half never leaves the part. Otherwise inject one at manufacture.
2. Register the public key, or have a certificate signed. The serial number of the part is the identity everything else is keyed on.
3. Store it where application code cannot leak it: a dedicated partition, with flash encryption on.
4. The broker maps that credential to an account and an ACL.

Wi-Fi is separate

The user's network credentials arrive at set-up time, over BLE or a temporary access point, and are stored next to the identity. They are not the identity.

Rotating credentials and the ACL

```
# mosquitto ACL file
user sensor01
topic write mec/dev/sensor01/#
topic read  mec/cmd/sensor01/#
```

A credential with no expiry is a credential you will never replace. Give it a lifetime you can actually meet, and build the renewal path before the first board ships.

Rotation has to be interruptible. The device keeps the old credential until the new one has completed a handshake, then discards it.

An ACL limits the blast radius. Revocation lists do not work well on devices, so rely on short lifetimes plus a deny list at the broker. Even a stolen device key should then only be able to publish under its own prefix.

Over the air: two slots and a bootloader

PARTITION	HOLDS	WRITTEN BY
bootloader	the code that picks a slot and verifies it	the factory only
app A	one application image	the updater, while B runs
app B	the other application image	the updater, while A runs
ota data	which slot to boot, and whether it is confirmed	the running application
settings	Wi-Fi credentials, identity, counters	the application

Plan the flash for two applications on day one. The device never overwrites the image it is running. It fills the inactive slot, verifies it, flips one pointer and reboots.

Rollback and signing

ROLLBACK

The new image boots and runs a self test: network up, sensor readable, broker reachable. Only then does it mark itself confirmed. Until that happens a watchdog reset sends the bootloader back to the previous slot. A version counter that only moves up stops an attacker replaying an old, vulnerable image.

SIGNING

The build signs the image with a private key that stays on the build machine. The bootloader verifies it against a public key held in the chip, and secure boot makes that check impossible to skip. The transport is not the trust boundary: verify the image even when it arrived over TLS.

An unsigned update path is remote code execution with a nice name. Whoever can answer the update request owns every board you shipped, whether or not they ever touched one.

Topic design for your project

```
mec/<team>/<device>/telemetry/<sensor>   device publishes, QoS 1
mec/<team>/<device>/status               retained, and the last will
mec/<team>/<device>/cmd/<name>           app publishes, device listens
mec/<team>/<device>/cmd/<name>/ack       device answers, same request id
```

Put identity in the topic and values in the payload. Keep the levels short, because every publish carries the whole string. Give each device exactly one retained status topic, and point its last will at that topic so the app gets a presence indicator for free.

Write the topic tree down before you write the firmware. It is the contract between the two halves of your project, and changing it later means reflashing a board and editing an app on the same afternoon.

RECAP

Three things to remember

1. MQTT is publish and subscribe through a broker. The device never learns who is listening, and a retained message plus a last will give the app a status topic it can trust.
2. Quality of service is per hop, not end to end. QoS 1 plus an idempotent payload with a sequence number is the right default for telemetry.
3. Identity and updates are part of the design. One credential per device over TLS, and a signed image in a second partition with a rollback that actually works.

FURTHER READING

mqtt.org · mosquitto.conf reference · RFC 7252, CoAP · pub.dev/packages/mqtt_client

NEXT

Before Lab 2

A broker running

Mosquitto on your laptop, anonymous access off, and mosquitto_sub printing what mosquitto_pub sends.

A sensor reading

The I2C sensor from Lecture 3 printing a value to the serial console every two seconds.

An app ready to listen

mqtt_client added to the pubspec, and a BLoC event for a reading arriving on a stream.

ALSO THIS WEEK

Agree the topic tree inside your team and write it into the repository README, so both halves of Lab 2 are built against the same names.

Questions?

dinu_stefan.rusu@upb.ro · Microsoft Teams: course channel · rusudinu.com